排版的本质，是把文字放进一个有限的框里，同时让读者感觉不到这个框的存在。字与字之间要呼吸，行与行之间要透气，标点则是呼吸里的停顿—短促、准确，不能含糊。好的版面不会让人注意到它的存在，只让人读得下去。反过来，糟糕的版面会不断提醒你：这里挤了，那里断了，这一行的最后一个字被硬生生拆成了两半。读者说不出哪里不对，但眼睛会累，读完一段就想放下。排版要做的，就是把这些“不对”提前消掉。这不是审美问题，而是计算问题：每一个字占多宽、每一行能放几个、断在哪里、剩下多少空间，都有数字可查。把这些数字算准，版面才有资格谈好看。数字不会说谎，版面也不会。算准了，才轮得到审美。这也是排版要先算后看的原因。

中文排版尤其如此。一个逗号的位置、一处引号的开合，都会改变整段文字的节奏。行首不能出现逗号与句号，行尾不能留下开引号；连续标点要挤掉一半的宽度，中英文之间要留出四分之一个字的空间。规矩只有几行，做起来却处处是分岔。每一条规则都在和另一条规则商量：避头尾要求把标点拉回来，挤压又要求它只占半格；断行想多放一个字，对齐又嫌这一行太满。最后落在纸上的，是一个可以被解释的结果，而不是一次碰巧。解释得清楚，规则才能被信任；解释不清楚，再漂亮的版面也只是运气。规则是死的，版面是活的，中间那层判断才是引擎的价值。

浏览器其实也在做同样的事，只是它从不解释。你把文字交给它，它给你一个结果；换一个浏览器，结果又不一样。于是同一份周报，在设计师的屏幕上读起来舒展，到了运营的手机上就变得拥挤。更麻烦的是，你没有办法问它“为什么断在这里”，它也不会告诉你哪一条规则起了作用。它把排版藏进黑盒，只留下一个看似理所当然的页面。对于普通网页，这已经足够；对于要导出、要印刷、要投到大屏上的文字，这远远不够。你需要知道每一行的余量，需要知道哪一个标点被压过，需要知道换一个宽度之后，哪些段落会重新变脏。这些数字，才是排版真正的手艺。

所以排版引擎要做的第一件事，不是把字画出来，而是把“为什么”留下来。每

一个断点都应该带着候选、代价和拒绝的理由；每一次挤压都应该能指出被压的是哪一个标点；每一行的余量都应该能被看见。诊断不是额外的装饰，它是排版结果的一部分，是这套系统愿意被检验的证据。当用户点开一行，看到“43 是逗号，不能放在行首，断点前移到 42”，他才会相信这一页不是碰巧排出来的。这份信任，比任何视觉效果都值钱。反过来，一个只会输出结果、不肯解释过程的引擎，永远只能被当成黑盒，出了问题也无从下手。解释不是辩解，而是把选择摊开给人看。

度量是这一切的地基。字宽、字高、基线、行距，看上去都是小数，累加起来却决定了一整页的形状。字体没有加载完就测量，整篇宽度都会错；布局阶段随手取整，四十个字就能差出一个像素。度量必须缓存，缓存必须可失效，失效之后必须重排——这三件事少一件，后面的断行和导出都会失真。更隐蔽的是空格：某些字体里空格的宽度是零，直接测量会得到零，必须用“中 中”减去两个汉字宽度来兜底。这些细节不会出现在效果图里，却会在导出时一次性暴露。所谓“像素一致”，从来不是最后一步调出来的，而是每一步都不肯将就的结果。度量稳了，后面的每一步才有意义。

断行的本质是选择。贪心策略只看眼前：这一行还能放下几个字，就在最后一个合法断点处停下。它快，也足够好，但每一行都在为自己的局部最优付出代价。Knuth-Plass 把整段文字看成一个整体，用动态规划寻找总代价最小的断点序列；它更慢，却能让行末的参差变得均匀。两种策略没有高下，只有场合：实时拖宽度时，贪心是唯一跟得上手指的选择；导出定稿时，多花几十毫秒换一页更稳的版面，完全值得。引擎要做的，是把两种结果都摆在桌面上，让人自己选，而不是替用户宣布哪一种才是“正确”。选择被记录下来，才有资格谈可复现。

避头尾是中文排版里最容易被忽略、也最容易被看见的规则。行首不能出现句读和闭括号，行尾不能留下开括号；遇到这种情况，断点要往前挪，或者把标点压窄半格。麻烦在于，挪动断点会改变这一行能放下的字数，压窄标点又会改变行宽，两个

动作互相牵制，写不好就会来回振荡。所以顺序必须是先决定断点，再处理标点，最后重新对齐；每一步只改一件事，结果才可复现。规则冲突不可怕，可怕的是冲突没有被记录下来。只要诊断里能说清“谁让了谁一步”，再复杂的版面也能被复盘。复盘得多了，规则才会变成经验。

标点挤压是另一种“让版面呼吸”的办法。行首的开括号、行尾的闭括号、连续出现的句读，都可以只占半个字宽。挤压不能改度量，只能在生成字形时改实际占宽；挤压也不能一边断行一边做，否则刚压完又放得下，放得下又不必断，断点就会跳来跳去。顺序必须是先断、后挤、再对齐。挤压还要克制：只压该压的标点，不要把整行都压薄；只压半格，不要压到看不清。留白是为了让字更好读，不是为了把字塞进框里。这条边界，靠对比图来守，也靠每一个被压过的标点留下记录。挤压的记录，也应该出现在诊断里。

中西混排的间距，是中文排版里最微妙的四分之一个字。汉字与拉丁字母、汉字与数字之间，需要一点额外的空气；标点两侧则不需要。这个间距不能写死在字体里，只能由引擎在生成字形时加上。加得太多，版面会散；加得太少，英文单词会像被汉字吞掉。尺度感来自反复对比，而不是某一条公式。把“中文ABC123”和“中文 ABC 123”并排放在一起，肉眼能看出差别，才算调对了。差别不需要夸张，刚好让人读得顺就够了。真正难的是让这份“刚好”在所有字号、所有字体下都成立。字号变了，间距也要跟着变，而不是照搬一个固定值。

对齐是最后一道工序。两端对齐要把行末的余量均匀分到字与字之间，但不能拉伸拉丁单词的内部，也不能让最后一行跟着一起撑开。余量太大时，宁可退回左对齐，也不要把一行字拉成稀疏的栅栏。居中和左对齐看起来简单，真正难的是知道什么时候不该对齐—这需要引擎给出余量的数字，而不是凭眼睛猜。间隙超过半个字，就该亮红灯；这是规则，也是分寸。对齐不是把每一行都填满，而是让每一行的

节奏一致。最后一行尤其要克制：它是段落的收尾，不是标尺的终点。分寸感，最后要变成可检查的数字。

导出是排版结果的第二次考试。预览里看到的每一行、每一个标点、每一处间距，都必须在导出的图片或 PDF 里原样出现。画布的像素比、离屏重绘的尺寸、字体是否就绪，任何一处不一致，都会让“所见即所得”变成一句空话。最稳妥的做法，是让预览和导出走同一条布局管线，只在最后一步换画布。预览可以用屏幕的像素比，导出可以用两倍或三倍；但字形的位置、断点的选择、挤压的宽度，必须来自同一份结果。否则，用户放大一看，就会发现预览骗了他。像素一致不是一句宣传语，而是一条可以被逐点验证的工程约束。把这条约束写进测试，比写进文档更可靠。

大文档把上面的问题放大了一百倍。十万字不能一次排完，要按段落切块、独立缓存、只重排脏段；排版要放进 Worker，主线程才能保持滚动流畅；虚拟列表只画视口里的行，剩下的等滚到再说。性能不是靠某一个技巧，而是靠“不重复做已经做过的事”。当规则改变时，只让受影响的段落变脏，这套系统才算真正跑起来。到了这一步，排版引擎才配得上“引擎”两个字。而在那之前，先把一行字画准，把一次宽度变化响应快，把一个断点的理由说清楚。所有宏大的架构，最后都要落在这三件小事上。这三件小事，决定了后面所有大事的上限。先把这一行画准，再谈下一行。